

Deep-AE — Phase 1 Project Documentation

1. Product Requirements Document (PRD)

Project Name

Deep-AE

Project Type

AI-Based Acoustic Structural Damage Detection System

Project Domain

- Artificial Intelligence
 - Audio Processing
 - Structural Health Monitoring (SHM)
 - Edge AI
 - Civil Engineering
-

2. Project Overview

Deep-AE is an AI-powered acoustic monitoring system designed to detect possible structural damage by analyzing sound patterns generated from materials under stress.

The system captures acoustic signals using microphones or acoustic sensors and processes them using Deep Learning models.

The AI model classifies structural conditions into categories such as:

- Healthy
- Crack Detected
- Severe Crack
- Environmental Noise

The primary focus of Phase 1 is to build and validate the core AI model.

3. Problem Statement

Traditional structural inspection systems are:

- manual,
- periodic,

- expensive,
- and incapable of continuous real-time monitoring.

Small structural cracks often generate unique acoustic signatures before visible damage occurs.

This project aims to create an AI system capable of detecting those acoustic anomalies automatically.

4. Main Objective

Develop a CNN-based audio classification system capable of:

- processing acoustic signals,
 - generating Mel Spectrograms,
 - and classifying structural conditions in real time.
-

5. Scope of Phase 1

Phase 1 ONLY focuses on:

- Dataset creation
- Audio preprocessing
- Mel Spectrogram generation
- CNN model development
- Training pipeline
- Evaluation pipeline
- Offline prediction system

Phase 1 DOES NOT include:

- Cloud deployment
 - IoT dashboard
 - Sensor arrays
 - Thermal integration
 - Vibration integration
 - Mobile application
 - Real-time edge deployment
-

6. Functional Requirements

FR-1 Audio Input

The system must accept WAV audio files.

FR-2 Audio Preprocessing

The system must:

- normalize audio,
- trim silence,
- resample audio,
- and generate spectrograms.

FR-3 Spectrogram Generation

The system must convert audio signals into Mel Spectrogram images.

FR-4 Model Training

The system must train a CNN model on labeled spectrogram data.

FR-5 Prediction

The system must classify audio into predefined classes.

FR-6 Evaluation

The system must generate:

- accuracy,
 - precision,
 - recall,
 - F1-score,
 - confusion matrix.
-

7. Non-Functional Requirements

NFR-1 Free Technology Stack

All tools and frameworks must be freely available.

NFR-2 Google Colab Compatibility

The entire training pipeline must run on Google Colab.

NFR-3 Modular Architecture

The codebase must be modular and scalable.

NFR-4 Lightweight Design

The CNN model should be lightweight enough for future edge deployment.

8. Success Criteria

The project will be considered successful if:

- The CNN model achieves acceptable classification performance.
 - The model can differentiate crack sounds from noise.
 - The preprocessing pipeline works consistently.
 - The prediction system correctly classifies unseen samples.
-

9. User Stories

User Story 1

As a developer, I want to upload structural sound recordings, so that the system can analyze them.

User Story 2

As a researcher, I want the system to classify crack sounds automatically, so that I can identify structural anomalies.

User Story 3

As a future deployment engineer, I want a lightweight model, so that it can later run on edge devices.

10. Technical Constraints

- Must use Python.
 - Must use PyTorch.
 - Must run on free GPU resources.
 - Must support WAV format.
 - Must support future real-time inference.
-

11. Assumptions

- Initial dataset will be manually collected.
- Environmental noise exists in recordings.
- Dataset size will initially be small.

- Data augmentation will be necessary.

12. Risks

Risk	Mitigation
Small dataset	Data augmentation
Noise interference	Filtering + preprocessing
Overfitting	Regularization + augmentation
Low-quality recordings	Better preprocessing

13. Deliverables

Phase 1 Deliverables:

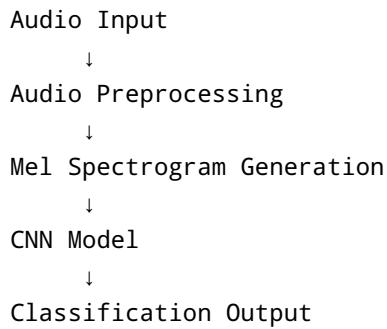
- preprocess.py
- spectrogram_generator.py
- dataset_loader.py
- model.py
- train.py
- evaluate.py
- predict.py
- trained_model.pth
- requirements.txt
- README.md

14. Technology Stack

Purpose	Technology
Programming Language	Python
AI Framework	PyTorch
Audio Processing	Librosa
Visualization	Matplotlib
Numerical Processing	NumPy
Data Handling	Pandas
Environment	Google Colab
Version Control	GitHub

15. System Architecture Document

High-Level Architecture



```
graph TD; A[Audio Input] --> B[Audio Preprocessing]; B --> C[Mel Spectrogram Generation]; C --> D[CNN Model]; D --> E[Classification Output];
```

Audio Input
↓
Audio Preprocessing
↓
Mel Spectrogram Generation
↓
CNN Model
↓
Classification Output

16. Detailed Pipeline

Stage 1 — Audio Collection

Input audio sources:

- metal impact sounds,
- crack sounds,
- structural vibration sounds,
- environmental noise.

Recording devices:

- phone microphone,
- USB microphone,
- piezoelectric sensor.

Stage 2 — Preprocessing

Operations:

- Convert to WAV
 - Resample to 22050 Hz
 - Trim silence
 - Normalize audio
 - Noise reduction (optional)
-

Stage 3 — Spectrogram Generation

Convert waveform into:

- Mel Spectrogram

Recommended parameters:

Parameter	Value
Sample Rate	22050 Hz
n_fft	2048
hop_length	512
n_mels	128
Image Size	128x128

Stage 4 — Dataset Creation

Dataset structure:

```
dataset/  
  healthy/  
  crack/  
  severe_crack/  
  noise/
```

Stage 5 — CNN Training

Training flow:

```
Spectrogram Image  
  ↓  
CNN Layers  
  ↓  
Feature Extraction  
  ↓  
Classification
```

Stage 6 — Prediction

The model outputs prediction probabilities.

Example:

```
{
  "healthy": 0.91,
  "crack": 0.07,
  "severe_crack": 0.01,
  "noise": 0.01
}
```

17. Dataset Specification Document

Dataset Type

Audio Classification Dataset

18. Dataset Classes

Class Name	Description
healthy	Normal structural sound
crack	Minor crack sound
severe_crack	Large fracture sound
noise	Environmental noise

19. Audio Specifications

Property	Value
Format	WAV
Sample Rate	22050 Hz
Channels	Mono
Duration	2 seconds
Bit Depth	16-bit

20. Dataset Collection Guidelines

Healthy Sounds

Examples:

- ambient room sound,
- stable vibration,
- low-noise structural sounds.

Crack Sounds

Examples:

- snapping,
- bending metal,
- cracking concrete,
- stressed material sounds.

Severe Crack Sounds

Examples:

- loud fractures,
- sudden snapping,
- high-energy break sounds.

Noise Sounds

Examples:

- traffic,
- footsteps,
- wind,
- speech,
- machinery.

21. Data Augmentation Requirements

Required augmentations:

- noise injection,
- time shift,
- volume scaling,
- pitch shift.

Purpose:

- improve robustness,
 - reduce overfitting,
 - simulate real-world environments.
-

22. Dataset Split

Recommended split:

Dataset	Percentage
Training	70%
Validation	15%
Testing	15%

23. Model Specification Document

Model Name

Deep-AE CNN v1

24. Model Objective

Build a CNN capable of classifying Mel Spectrogram images.

25. Input Specification

Input Type:

- Mel Spectrogram image

Input Shape:

128 × 128 × 1

26. Output Specification

Output Type:

- Multi-class classification

Classes:

- healthy
- crack
- severe_crack
- noise

27. CNN Architecture

```
graph TD;
    A[Input Layer] --> B[Conv2D];
    B --> C[ReLU];
    C --> D[MaxPooling];
    D --> E[Conv2D];
    E --> F[ReLU];
    F --> G[MaxPooling];
    G --> H[Flatten];
    H --> I[Dense Layer];
    I --> J[Dropout];
    J --> K[Output Layer];
```

28. Recommended Hyperparameters

Parameter	Value
Optimizer	Adam
Learning Rate	0.001

Parameter	Value
Batch Size	16
Epochs	30
Loss Function	CrossEntropyLoss
Dropout	0.3

29. Evaluation Metrics

Required metrics:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Priority metric:

Recall.

Reason: Missing a crack is more dangerous than false alarms.

30. Development Roadmap

Week 1

- Setup environment
- Learn audio preprocessing
- Organize project structure

Week 2

- Collect dataset
- Label recordings
- Prepare WAV files

Week 3

- Generate Mel Spectrograms
- Build preprocessing pipeline

Week 4

- Build CNN model
- Implement training pipeline

Week 5

- Train model
- Evaluate model
- Improve accuracy

Week 6

- Build prediction pipeline
 - Test on unseen audio
-

31. Folder Structure

```
Deep-AE/
|
├── dataset/
|   ├── healthy/
|   ├── crack/
|   ├── severe_crack/
|   └── noise/
|
├── spectrograms/
|
├── models/
|
├── notebooks/
|
├── src/
|   ├── preprocess.py
|   ├── spectrogram_generator.py
|   ├── dataset_loader.py
|   ├── model.py
|   ├── train.py
|   ├── evaluate.py
|   └── predict.py
|
├── requirements.txt
├── README.md
└── config.yaml
```

32. requirements.txt Specification

Required libraries:

```
torch
torchaudio
librosa
numpy
pandas
matplotlib
scikit-learn
opencv-python
soundfile
noisereduce
```

33. README Specification

README must contain:

- project overview,
- setup instructions,
- dataset structure,
- training instructions,
- prediction instructions,
- future roadmap.

34. Claude Prompt Template

Use this exact style when asking Claude to generate code.

```
You are a senior ML engineer.
```

```
Build a complete CNN-based audio classification pipeline
for structural crack detection.
```

```
Requirements:
```

- ```
- Python
- PyTorch
- Google Colab compatible
- WAV audio input
- Mel Spectrogram preprocessing
- Multi-class classification
```

Generate:

1. preprocess.py
2. spectrogram\_generator.py
3. dataset\_loader.py
4. model.py
5. train.py
6. evaluate.py
7. predict.py
8. requirements.txt
9. README.md

Constraints:

- Modular code
- Clean architecture
- Beginner-friendly
- Well-commented
- Production-ready structure

---

## 35. Future Upgrade Notes

Future upgrades may include:

- real-time microphone monitoring,
- edge deployment,
- Raspberry Pi integration,
- transformer models,
- multi-modal sensor fusion,
- thermal analysis,
- vibration analysis.

These are NOT part of Phase 1.

---

## 36. Final Goal of Phase 1

The final output of Phase 1 should be:

```
Audio Input
 ↓
Mel Spectrogram
 ↓
CNN Prediction
 ↓
Healthy / Crack / Severe Crack / Noise
```

This completes the core AI engine of Deep-AE.